

Atlys

A Connected Research Knowledge Management System

Authors

Matt ter Steege
Matttersteege@gmail.com

Contents

1	Problem and Proposal	2
2	Vision and Differentiation	2
2.1	Comparison to Existing Tools	2
3	Research Question and Evaluation	3
3.1	Hypotheses	3
3.2	Study Design	3
3.3	Materials and Task Corpus	3
3.4	Procedure	3
3.5	Outcome Measures	4
3.6	Analysis Plan	4
3.7	Pilot Study	4
4	Requirements	4
4.1	Functional Requirements	5
4.2	Non-Functional Requirements	5
5	Personal Learning Objectives	6
6	Timeline	6
6.1	Period 1 — Building the retrieval core	6
6.2	Period 2 — Proving the retrieval core	6
6.3	Period 3 — Synthesis	6
6.4	Period 4 — Writing Integration	6
6.5	Explicitly Out of Scope for this project	7
7	Risk Analysis	7

1 | Problem and Proposal

Researchers and students accumulate large personal libraries of papers, notes, videos, and clippings, but existing tools organise this material around files and folders rather than around ideas. Reference managers (Zotero, Mendeley, EndNote) track citations well but say nothing about content. Note apps (Notion, Obsidian) capture content but disconnect it from formal citation and writing workflows. As a result, the recurring bottleneck in academic writing is not a lack of sources — it is not being able to find, at the moment of writing, which of the sources already read supports the statement you want to write.

Across a literature review or thesis, a student typically reads a whole lot of sources. Existing tools fail at four connected points:

- **Retrieval is filename- or folder-based, not idea-based** — locating “that paper about self-determination theory” means remembering which folder it was filed in, not what it said.
- **Notes are disconnected from their source location** — a highlight or annotation in a PDF reader rarely survives as a linked, searchable object once the student moves to writing.
- **The writing step requires context-switching** — moving between a reference manager, a notes app, and a word processor to assemble a single well-cited paragraph.
- **Sources contain a lot of fluff** — the sources read usually contain a lot of unneeded text, but those are always present when searching.

Each problem propagates to the next problem. Because retrieval is weak, notes get re-written from scratch. Because notes are disconnected, writing requires re-reading sources. The result is that literature review time scales poorly with library size.

Atlys addresses this by replacing the “file” as the unit of organisation with the “**chunk**” — a paragraph, highlight, note, or (self-annotated) figure — linked back to its source and tagged with the concepts it relates to. Papers, videos, and web pages become containers of (one or more) chunks rather than being the searchable unit itself. This enables semantic search across an entire personal library (“find everything I’ve read about intrinsic motivation”), regardless of which file, page, or video timestamp the material originally came from.

This proposal scopes a full academic year of work, divided into two terms. Term 1 (Periods 1–2) is dedicated to building and evaluating the retrieval foundation. Term 2 (Periods 3–4) is dedicated to layering synthesis and writing-integration features on top.

2 | Vision and Differentiation

Atlys is positioned as a *research knowledge management system* (RKMS), not (only) a reference manager. The distinguishing claim is architectural: every object in the system — a paper, a highlighted passage, a video clip, a personal note — is represented as a first-class, independently searchable and linkable unit, connected through a concept graph rather than a folder hierarchy.

Comparison to Existing Tools

Tool	Strength	Gap
Zotero / Mendeley / EndNote	Citation management, metadata	No semantic search over content; notes are isolated per-item
Notion / Obsidian	Flexible note linking	No native citation model or PDF/reference extraction
Elicit	Semantic search over published literature	Searches the world’s papers, not the user’s own annotated library
Recall	Chunk-level capture and graph linking	General-purpose; no academic citation, claim-evidence, or writing-integration workflow

The differentiating combination is: (1) chunk-level semantic search, (2) scoped specifically to the user’s own reading library, (3) integrated with an academic writing workflow (citation and quote insertion) and (4) built around a claim/evidence model suited to argumentative academic writing. No single existing tool combines all four.

3 | Research Question and Evaluation

Alongside the application itself, this project delivers empirical evidence for whether chunk-level semantic search actually helps, rather than asserting it by construction. The research question is:

Does semantic, chunk-level search over a personal research library reduce time-to-locate and improve recall accuracy, compared to folder- and filename-based reference management, for students conducting a literature review?

Hypotheses

The research question is decomposed into three testable hypotheses, each with a corresponding null:

- **H1 (speed):** Mean time-to-locate for a source-finding task is lower with Atlys than with the baseline tool. ($H1_0$: no difference in mean time-to-locate.)
- **H2 (accuracy):** The proportion of tasks answered correctly is higher with Atlys than with the baseline tool. ($H2_0$: no difference in accuracy.)
- **H3 (usability):** The post-task SUS score for Atlys is higher than for the baseline tool. ($H3_0$: no difference in SUS score.)

H1 and H2 are the primary hypotheses; H3 is secondary and reported for context even if H1/H2 are not confirmed.

Study Design

Yes, I definitely used ChatGPT to find figure out how to shape the research, so if a different way of research might fit better, it is open to change!

A **within-subject** design is used: every participant completes tasks in both the *baseline* condition (Zotero, with sources filed into a fixed folder/tag structure, no semantic search) and the *Atlys* condition (same sources, chunk-level semantic search), so that between-participant variance in reading speed or domain knowledge cannot distort the comparison results. Participants are randomly assigned to one of two orderings (baseline-then-Atlys or Atlys-then-baseline) to control for learning and fatigue effects, and each ordering uses a different, difficulty-matched task set (Set A / Set B), so no participant sees the same question twice. This gives a 2×2 Latin-square structure (order $\times$ task set) with $n \approx 12$ –20 participants, recruited from students who have done at least one literature review (e.g. a prior thesis or research course), since the target population is literature-review work, not general search.

Materials and Task Corpus

Both conditions are pre-loaded with an **identical corpus** of 40–50 papers drawn from a topic area familiar to CS/AI bachelor students (e.g. human-computer interaction), so that no participant is search-limited by unfamiliar terminology. Before the study, the corpus is:

- filed into a realistic, non-trivial folder/tag hierarchy for the baseline condition (mirroring how a student would actually file 40+ PDFs, not an artificially easy structure);
- chunked and embedded for the Atlys condition, using the same highlights/notes in both conditions so the *content* available is equal and only the *retrieval mechanism* differs.

An **answer key** is constructed in advance: for every task, the specific source and passage that correctly answers it is identified by me. This is what makes “retrieval accuracy” a scoreable, non-subjective outcome rather than a judgement call made during the study.

Two task sets (A/B) of 8–10 tasks each are constructed, difficulty-matched pairwise (e.g. Task A3 and Task B3 both require locating a single specific claim across the corpus). Each task is phrased as a natural research question a student would actually ask, e.g. “*Find a source that argues extrinsic rewards can undermine intrinsic motivation, and identify the specific paragraph making that claim.*” Tasks are reworded using vocabulary not directly found in the target passage, so that success requires genuine retrieval rather than keyword matching.

Procedure

1. **Consent and background survey** (5 min): prior literature-review experience, familiarity with Zotero/reference managers.

-
2. **Onboarding** for whichever tool comes first (≤ 10 min, per requirement N5), ending with one practice task not counted in results.
 3. **Condition 1**: 8–10 timed tasks. For each task, the timer starts when the task prompt is shown and stops when the participant submits the source + passage they believe answers it, with a 5-minute timeout recorded as a censored (not zero) time.
 4. **Post-condition SUS questionnaire** for condition 1.
 5. **Short break**, then onboarding + tasks + SUS for condition 2, mirroring steps 2–4.

Outcome Measures

- **Time-to-locate** (H1): seconds from task display to submission, per task, per participant. Timeouts are treated as right-censored rather than dropped.
- **Retrieval accuracy** (H2): binary correct/incorrect per task, scored against the pre-built answer key; a submission is correct if it identifies the correct source *and* a passage within one paragraph of the annotated answer.
- **Usability** (H3): the standard 10-item SUS per participant per condition.
- **Qualitative**: interview notes, coded for recurring breakdown themes (e.g. “didn’t trust top result”, “didn’t know how to phrase the query”).

Analysis Plan

Because the design is within-subject, each participant serves as their own control, and outcomes are compared *paired* rather than between groups:

- **H1**: per-participant median time-to-locate (baseline vs Atlys) compared with a paired Wilcoxon signed-rank test, effect size reported as matched-pairs rank-biserial correlation.
- **H2**: accuracy is repeated binary data per participant, analysed with McNemar’s test on the paired correct/incorrect counts.
- **H3**: paired *t*-test on per-participant SUS totals.
- Significance level $\alpha = 0.05$ for all three, reported alongside effect sizes and confidence intervals rather than *p*-values alone, given the modest *n*.

A sensitivity/power note is included in the final report: with $n = 12$ – 20 paired observations, the study is powered to detect a medium-to-large effect ($d \approx 0.6$ – 0.8); a null result is therefore reported as “no detectable effect of this size” rather than “no effect”, and the report states this limitation explicitly.

Pilot Study

Before recruiting the full sample, 2–3 pilot sessions are run internally to (a) confirm no task in Set A/B is a floor or ceiling case (all-succeed or all-timeout), (b) verify the 5-minute timeout and ≤ 10 -minute onboarding are realistic, and (c) sanity-check that the answer key is unambiguous to someone who is not the researcher. Task wording and timing are adjusted based on pilot results before Period 2 data collection begins.

This design is achievable within the two-term timeline because it depends only on the Period 1–2 retrieval MVP (Requirements F1–F6, N1–N3 in Section 4), not on the Period 3–4 writing-integration features.

4 | Requirements

The requirements below translate the vision (Section 2) and the two-term plan (Section 6) into concrete, verifiable statements, prioritised using MoSCoW. Requirements are grouped by the term in which they become relevant, so that the priority list can be read directly against the timeline.

Functional Requirements

ID	Requirement	Priority
F1	The system shall ingest a PDF file and extract text, headings, and reference metadata.	Must
F2	Users shall be able to manually highlight a passage in an uploaded PDF to create a Chunk.	Must
F3	The system shall generate a vector embedding for every Chunk.	Must
F4	The system shall accept a natural-language query and return a ranked list of Chunks by semantic similarity.	Must
F5	The system shall expose CRUD operations on Source, Chunk, Concept, Project, and Claim via the Core API.	Must
F6	Users shall be able to browse their library and view a Source with its associated Chunks.	Must
F7	The system shall export a Project's Sources to a BibTeX file.	Should
F8	Users shall be able to tag a Chunk with a Concept, optionally specifying the relation (mentions/supports/contradicts).	Should
F9	The system shall generate a Concept page that aggregates every Chunk linked to it.	Should
F10	A browser extension shall allow clipping a webpage or YouTube timestamp directly into a Chunk.	Could
F11	Users shall be able to attach a Chunk to a Claim as supporting or contradicting evidence, with a confidence score.	Could
F12	A search-and-insert panel shall allow inserting a cited quote into an external document while writing.	Could
F13	A Word add-in shall provide the search-and-insert panel inside Microsoft Word.	Won't (this year)

Non-Functional Requirements

ID	Requirement	Priority
N1	A semantic search query shall return ranked results in under 2 seconds for a library of up to 2,000 chunks, so the user study is not confounded by system latency.	Must
N2	All user-uploaded documents and derived data (chunks, embeddings) shall be stored on infrastructure under the user's/researcher's control, with no third-party API receiving document content, to keep the self-hosted design GDPR-compatible.	Must
N3	Authentication and authorisation shall be handled through the Evita IDP rather than custom credential storage.	Must
N4	The Core API shall remain usable (CRUD available) if the embedding/search layer is temporarily unavailable, i.e. search is a degradable, not blocking, dependency.	Should
N5	The system shall be usable by a participant with no prior training within the length of a single user-study session (≤ 10 minutes onboarding).	Should
N6	The codebase shall be structured so that the embedding/search layer can be swapped (e.g. Ollama model, or vector database) without changing the Core API contract.	Could

Requirements marked **Must** correspond to the Period 1–2 retrieval MVP (Section 6.1–6.2) and are the ones the research question in Section 3 depends on; **Should/Could** items correspond to Period 3–4 and can slip without invalidating the evaluation.

5 | Personal Learning Objectives

Beyond delivering the artifact described above, this project is scoped to develop the following skills, which are new to me relative to prior coursework:

- **Vector search implementation** — designing and implementing semantic (embedding-based) search over a real dataset, including choosing a chunking strategy and evaluating ranking quality, rather than only using search as a black-box library feature.
- **Running a self-hosted inference service** — deploying and operating Ollama in a production-like setup, including the operational trade-offs (latency, resource usage, model choice) that come with self-hosting rather than calling a managed API.
- **Sustaining a single system across a full academic year** — managing scope and technical debt across two terms on one codebase, rather than the shorter, disposable projects typical of regular coursework.
- **Data modelling for a domain with typed relationships** — designing a schema where the relationship itself (supports/contradicts/mentions) carries meaning, which requires different modelling judgement than a conventional CRUD schema.

6 | Timeline

Period 1 — Building the retrieval core

Goal: Create a crude system that allows users to store data and find data based on natural language.

- Data model implementation: Source → Chunk → Concept
- PDF ingestion pipeline: text extraction, heading/section detection, reference list parsing, figure/caption extraction
- Chunking strategy: paragraph-level splitting with page/offset anchoring
- Embedding generation and vector search (semantic query → ranked chunks)
- **Period 1 milestone:** Have a functioning API that allows CRUD (Create, Read, Update, Delete) to be made on data stored.

Period 2 — Proving the retrieval core

Goal: prove that chunk-level semantic search meaningfully outperforms file-based browsing.

- Web UI: upload sources, browse a library, highlight-to-chunk, run searches
- Simple export of the sources to a BibTeX file (or other relevant medium).
- **Period 2 milestone:** run the user study described in Section 3 and report results

Period 3 — Synthesis

Goal: extend the validated retrieval core into the workflow surrounding it.

- Concept pages that auto-aggregate every chunk referencing that concept
- Browser extension: clip web pages and YouTube timestamps directly into chunks
- Search-and-insert panel for assembling citations/quotes while writing (standalone panel first; a Word add-in via the Office JavaScript API is a stretch goal, not a dependency)
- **Period 3 milestone:** Have an workflow that is easily usable by users.

Period 4 — Writing Integration

Goal: extend the validated retrieval core into the workflow surrounding it, without re-litigating the foundation.

- Claim/evidence linking: attach chunks to a claim as supporting or contradicting evidence, with a confidence indicator
- Automated argument-mapping / visual claim graphs beyond a simple claim–evidence list
- **Period 4 milestone:** Be able to structure thought and satisfy the original idea of this project.

Explicitly Out of Scope for this project

- Full Word/Overleaf live add-ins (BibTeX export is sufficient; live add-ins are future work, but first in line may it be feasible to add)
- Multi-user real-time collaboration
- Mobile apps

7 | Risk Analysis

The risks below are ordered by how directly they threaten the Period 2 evaluation, since the research question (Section 4) is the highest-stakes deliverable and depends on the Period 1–2 work landing on time.

Risk	Likelihood	Impact	Mitigation
Fewer than 12 participants can be recruited for the within-subject study.	Medium	Medium	Recruit early and over-invite (aim for 20+ sign-ups); recruit from multiple year groups/programmes rather than a single cohort; fall back to a smaller- n study with the limitation noted explicitly in the report.
Self-hosted embedding quality (Ollama model) is insufficient for meaningful ranking.	Medium	High	Benchmark 2–3 candidate embedding models against a small hand-labelled query set before committing, during Period 1; keep the embedding layer swappable (Requirement N6) so a stronger model can be substituted without a redesign.
PDF text/reference extraction is unreliable across varied source formatting (scanned PDFs, multi-column layouts).	High	Medium	Scope Period 1 ingestion to well-formatted, text-based PDFs; explicitly exclude scanned/image-only PDFs from the user-study corpus rather than solving OCR this year.
Scope creep from Period 3–4 features (claim graphs, browser extension, Word add-in) delays the core evaluation.	Medium	Medium	Requirements are prioritised with MoSCoW (Section 4); Must-items are strictly limited to what the research question needs, and Period 3–4 items are explicitly allowed to slip without affecting the Period 2 result.
Single-person delivery means no redundancy if illness/unavailability affects a period.	Low	High	Keep milestones granular (weekly, not per-period) so a slip is visible early; the Period 1 milestone (a working CRUD API) is deliberately minimal so later periods are not blocked on a large single milestone.